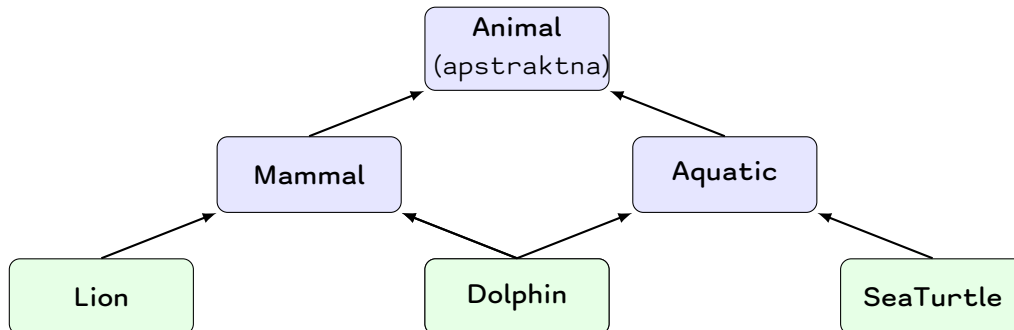


Objektno programiranje

vježba 9

višestruko nasljeđivanje, templates, exceptions, smart pointers

Napišite program za upravljanje životinjama u zooškom vrtu. Svaku klasu pišite u posebnu datoteku i odvojite implementaciju i interface.



1. Definirajte apstraktnu klasu `Animal` sa **čisto virtualnim funkcijama**:

- `string getSpecies() const` - vraća vrstu životinje
- `double getDailyFood() const` - vraća dnevnu količinu hrane u kg
- `string getName() const` - vraća ime životinje

U konstruktoru klase `Animal` izvršite validaciju podataka i bacite `invalid_argument` iznimku ako: ime je prazno, godine < 0 , težina ≤ 0 .

2. Izvedite klase prema dijagramu. Dodajte u klase specifične karakteristike:

- `Mammal`: član `hasFur (bool)`. Konstruktor `Mammal` treba primiti dodatni parametar `hasFur` i ispisati poruku o stvaranju sisavca (npr. "Stvoren sisavac s dlakom" ili "Stvoren sisavac s malo dlake").
- `Aquatic`: član `maxDiveDepth (double)`. Konstruktor `Aquatic` treba primiti dodatni parametar `maxDiveDepth` i ispisati poruku o stvaranju vodene životinje (npr. "Stvorena vodena životinja koja roni do Xm").

3. Svaka konkretna klasa u konstruktoru prima `name (string)`, `age (int)`, `weight (double)` i poziva odgovarajući konstruktor bazne klase s dodatnim parametrima:

- `Lion`: `hasFur = true`
- `Elephant`: `hasFur = false`
- `Dolphin`: `hasFur = true` i `maxDiveDepth = 300.0` (nasljeđuje i `Mammal` i `Aquatic`)
- `SeaTurtle`: `maxDiveDepth = 200.0`

Member funkcija `getDailyFood()` računa dnevnu količinu hrane kao postotak težine (lav: 6% težine, slon: 4% težine, dupin: 5% težine, morska kornjača: 3% težine) i baca `logic_error` ako je količina 0.

4. Napišite template klasu `ZooSection<T>` koja:

- Koristi `vector<unique_ptr<T>>` za spremanje životinja.
- Ima funkciju `void addAnimal(unique_ptr<T> animal)` - preuzima vlasništvo.
- Ima funkciju `double totalFood() const` - vraća ukupnu dnevnu hranu.
- Ima funkciju `size_t size() const` - vraća broj životinja.

- Ima funkciju `T* getAnimal(size_t index)` **const** - vraća raw pointer na životinju (koristite `ptr.get()` za dohvatiti raw pointer iz `unique_ptr`).
- Baca `runtime_error` ako se pokuša dodati `nullptr`.

5. Napišite klasu `ZooKeeper` koja:

- Ima funkciju **void** `processAnimal(Animal* animal)` koja prima pointer na životinju (bez preuzimanja vlasništva) i ispiše: "Species: [vrsta], Name: [ime], Food: [hrana]kg".
- Ima static član `totalAnimalsServed` za broj ukupno nahranjenih životinja.
- Ima static funkciju `getTotalAnimalsServed()` za pristup brojaču.

6. U main funkciji:

- Kreirajte životinje koristeći `make_unique` (jednog lava, slona, dupina i morsku kornjaču).
- Dodajte ih u `ZooSection<Animal>`.
- Pošaljite svaku životinju `ZooKeeperu` na hranjenje.
- Ispišite ukupnu dnevnu količinu hrane i broj nahranjenih životinja.
- Demonstrirajte hvatanje iznimki korištenjem **try-catch** blokova.

Napomene

- Za rješavanje diamond problema, klase `Mammal` i `Aquatic` moraju virtualno nasljeđivati `Animal`.
- Klasa `Animal` treba imati **virtual destruktor** zbog korištenja `unique_ptr<Animal>`.
- Konstruktor klase `Animal` treba biti `protected` jer je klasa apstraktna.
- Informacije o karakteristikama životinja (dlaka, dubina ronjenja) ispisuju se automatski prilikom stvaranja objekata.